ELEC-E5550 - Statistical Natural Language Processing D

Q-RAG: Improving Education QA with Query-to-query Retrieval and Feedback Loop

Final Report

My Nguyen - my.1.nguyen@aalto.fi
Hung Nguyen - hung.6.nguyen@aalto.fi

Aalto University

April 15, 2025

Contents

1 Introduction

Retrieval-Augmented Generation (RAG) [1] has become a popular way to give Large Language Models (LLMs) access to external knowledge without the cost of fine-tuning. By retrieving relevant documents and using them as context for each query, RAG improves response accuracy and relevance, making it especially useful for educational question-answering (QA) systems [2]. In an education setting, a RAG-based chatbot can pull information from textbooks, research papers, and instructional materials to provide precise, up-to-date explanations to students' queries. They can help students by providing quick access to relevant information, generating detailed explanations, and enhancing the learning experience. Additionally, they can reduce the workload for teaching assistants (TAs) and instructors by handling repetitive student questions, allowing educators to focus on more complex learning needs.

However, a major issue in these systems is the retrieval step, where the retrieved information is often incomplete or lacks the necessary details to answer a question properly [3]. This happens because queries and information chunks do not naturally exist in the same semantic space; in other words, queries and chunks can mismatch due to different language/word usage. This makes the query-to-chunk matching mechanism utilized in generic RAG unreliable. As a result, these models may often fail to retrieve the correct information, resulting in superficially relevant responses but ultimately unhelpful, which is a serious problem in education where accuracy matters.

Another notable issue is the lack of human-in-the-loop mechanisms [4]. While open-domain QA systems cannot realistically rely on human oversight, smaller-scale applications—like educational chatbots—can benefit significantly from expert intervention. Teachers and teaching assistants are the best sources for handling complex student questions that the chatbot alone cannot resolve. However, current RAG models do not effectively integrate human feedback, mainly because retrieval systems function like "black-boxes" with little room for direct control. This leads to a cycle where students keep getting the same poor responses, and instructors have to repeatedly step in to correct such misunderstandings.

To overcome these challenges, we propose Q-RAG, a novel approach that integrates structured human feedback into the retrieval process. Our method features two main improvements from the issues identified in generic RAG:

- Query-to-query Retrieval: Indexing course content by representative *model questions* to better match student queries.
- Feedback-Loop Learning: Allowing direct intervention to address poor performance, preventing the model from making similar mistakes.

The main goal of the project is to develop and evaluate Q-RAG within an educational context to show that this proof of concept can be an effective tool for both students and educators. At the same time, this research can help deepen our understanding of statistical NLP through in-depth and serious research regarding retrieval-based language modelling, which is of high interest in the community lately. Hopefully, this paper can sufficiently demonstrate not only our knowledge but also our strong enthusiasm for the subject. The codebase of the project can be found in Appendix A.

The following section presents a structured literature review. This review examines current approaches and identifies the research gap that Q-RAG addresses, which helps to contextualize our work. Section 3 introduces Q-RAG's theoretical framework, detailing its architecture and core principles. In Section 4, we describe the evaluation QA pairs that serve as our experimental dataset. Section 5 presents our experimental methodology, comparing Q-RAG against baseline models through an evaluation pipeline. The subsequent results analysis (Section 6) provides assessments of the experiment results. We conclude in Section 7 with a discussion of limitations and promising future research directions.

2 Literature Review

2.1 Retrieval Augmented Generation (RAG)

In this section, we provide background on RAG, tracing its evolution from the originally proposed model to a widely adopted paradigm for QA applications. We also discuss its broader challenges related to retrieval.

2.1.1 The Original RAG Model

Retrieval Augmented Generation (RAG) was proposed by Lewis et al. [1] as a method to address hallucinations caused by the lack of domain-specific knowledge of LLMs. In essence, RAG retrieves and augments an LLM agent's knowledge base with relevant context. Through in-context learning, the LLM can adapt to the given context and produce responses that are more relevant and grounded. This allows LLMs to adapt to a specific domain while mitigating the need for specialized training, which is especially useful in knowledge-intensive environments. The original RAG system proposed by the authors was shown to outperform several strong competitors in popular QA benchmarks, setting it as the state-of-the-art method in information-intensive domains at the time.

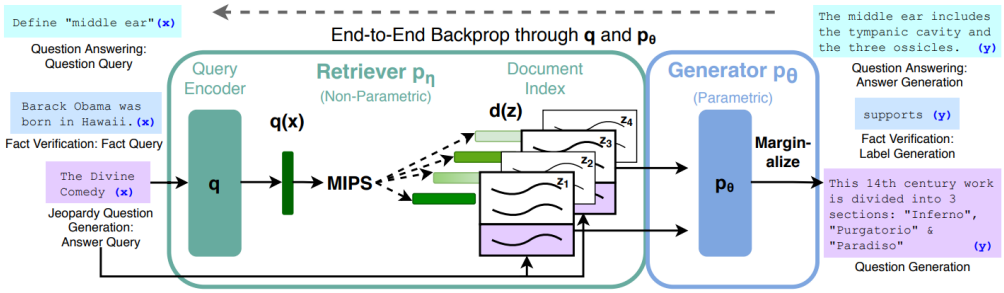

Figure 1: Lewis et al.'s proposed RAG architecture [1]

As illustrated in Figure 1, the original system contains two main components, the *retriever* and the *generator*. Overall, it behaves similarly to a standard encoder-decoder structure: upon receiving an input sequence x, the encoder transforms it into a dense embedding vector, which is later fed to the decoder to generate an output sequence.

To introduce the difference maker, before passing the encoder's output into the decoder, the retriever would search for K most similar documents to the query x. This is achieved using a vector search algorithm that maximizes the inner product between the query and document embedding vectors. The process continues by augmenting the most relevant documents z into the generator, allowing it to be conditioned on a specific context and produce responses with factual accuracy and relevance through in-context learning [5].

2.1.2 RAG as a Design Pattern

The original RAG system was built as an end-to-end trainable model, allowing the retriever integrated with the system to be tuned during training. Nevertheless, this process can be challenging to reproduce because of its time consumption and computational costs [6]. As such, nowadays, RAG is instead viewed as a design pattern to create an external knowledge base for LLM chatbots in QA use cases. This is especially important in knowledge-intensive environments where it is not possible to ingest a large amount of information into the LLM directly. In certain settings, various higher-level RAG-based frameworks with different techniques are designed, forming a large ecosystem [7].

Most RAG systems are constructed based on the Naive RAG architecture, which is illustrated in Figure 2. Naive RAG is a high-level framework with the most fundamental components of a RAG system: the retriever and the generator. The retriever consists of an embedding model to encode queries and documents into vector embeddings, along with a vector database storing document embeddings and performing similarity-based document retrieval. On the other hand, the generator can be any LLM agent suitable for text generation such as GPT models. The overall process is similar: the retriever component retrieves the top-K most relevant documents to the query, which are then augmented into the generator so that the LLM agent can be conditioned on a specific context. This helps the LLM to produce accurate and relevant responses.

This simple framework allows the main idea of RAG to be preserved while providing room for additional development. Retrieval-based offers a new way to augment LLM systems, previously with no specialized knowledge. This makes RAG particularly popular in QA applications [8], where

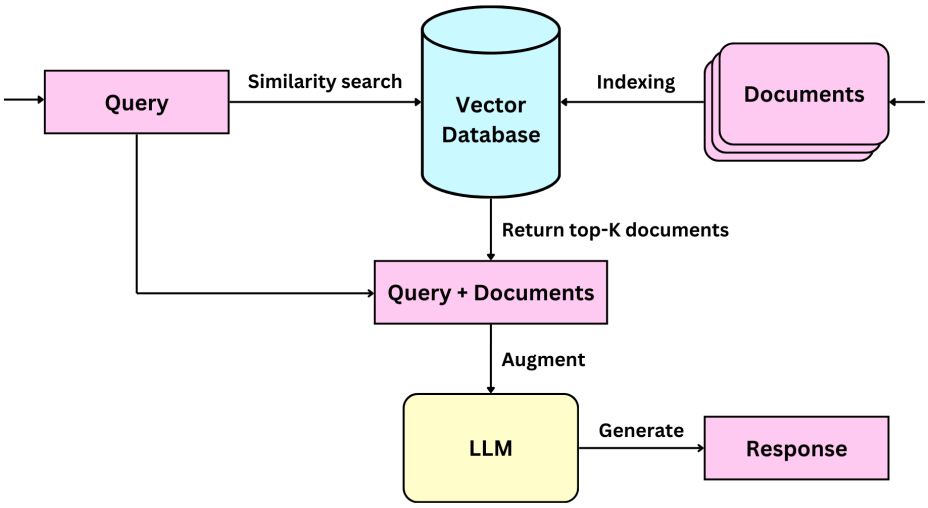

Figure 2: RAG as a design pattern. The retriever searches for relevant documents given the user query. These documents serve as context to help the LLM during in-context learning, therefore producing a more informed response.

it can provide more accurate, context-aware, and up-to-date responses while avoiding the need for fine-tuning. In practical scenarios, many advanced strategies can be applied to improve certain aspects of the original framework or to tune the system to a specific context.

2.1.3 The Broad Challenge of RAG

One of the primary challenges in RAG systems is retrieval quality - the ability of the retriever to search for the necessary information [3]. Naive RAG relies on embedding-based similarity matching between questions and chunks, where these elements are mapped into a shared vector space for matching. However, this approach often fails when queries and relevant documents lack lexical overlap. This issue arises because human-written queries and texts follow different linguistic styles, leading to mismatches in language use. In other words, questions and document chunks exist in different spaces, serving different communicative purposes; questions are typically concise and explicitly seek missing information, while document chunks are statements written without a specific question to address. As a result, they differ in structure and representation within the embedding space, making direct matching unreliable. The result of poor retrieval is superficially relevant chunks that lack the precise details needed to answer the original query.

There has been active research in the domain exploring retrieval improvements. One potential solution is query rewriting. This technique involved addressing the complexity involved in user-side input. RAG Fusion is one notable approach, proposed by Raudaschl A. [9]. This involves reformulating the input user query into smaller but simpler sub-queries. The author justified that by decomposing complex queries into more manageable sub-queries, it is possible to allow the model to focus on more targeted aspects of the question, which helps the model to align better with document chunks. Rackauckas et al. [10] reported that RAG Fusion was able to outperform the generic RAG model in terms of document recall rate, response accuracy, and comprehensiveness.

There has been a body of research focused specifically on improving the representations of chunks for better query-to-chunk alignment. Many existing RAG approaches were built upon the work of Dense Passage Retrieval (DPR) [11], which relies on dense embedding models to encode text into vectors for semantic matching. Nonetheless, some have argued that this document representation approach may lead to a focus on overall relevance rather than specificity, and thus harm retrieval accuracy. Hybrid RAG [12] addressed this by introducing sparse embeddings in addition to dense ones. Sparse retrieval methods, such as BM25 [13], preserve keyword-level precision. This approach can improve specificity by making sure the document contains information regarding the exact terms and entities. By combining sparse and dense retrieval, the authors showed that this approach outperformed generic dense retrieval in terms of document recall rate

in multiple challenging QA datasets.

Corrective RAG [14] is a notable approach designed to mitigate poor retrieval by implementing a fail-safe mechanism. When an LLM judge detects poor retrieval, Corrective RAG performs a web search to gather additional information to augment the content documents fed to the generator. The authors argued that this process helps to refine the retrieved information, especially considering that irrelevant context from poor retrieval can significantly degrade the quality of generated responses. The authors evaluated the models in a controlled web environment and discovered that this approach outperformed RAG baselines in terms of answer accuracy.

Existing solutions tackle the same retrieval challenge in different ways. Some attempted to "push" the representations on both sides (query and chunks) closer for better alignment, such as performing query expansion (RAG Fusion) so they have better chances to match chunks, or adjusting chunk representations (Hybrid RAG) so they have better chances to match queries. Others introduced "safety nets" as a fall-back mechanism to such failures, by improving chances for better recall by accessing a larger knowledge base. While they show promise in mitigating poor retrieval, these approaches primarily find a *work-around* for the mismatch rather than directly solving the fundamental mismatching issue. Query/chunk representations refinement may improve matching chances, but they will inherently fail because queries and chunks are represented for different communicative purposes.

2.2 RAG Chatbots in Education Context

In education, RAG allows conversational agents and tutors to ground their answers in trusted course materials, ensuring alignment with the curriculum and reducing hallucinations [15]. Common RAG use cases in education include administrative assistance, which facilitates access to university policies, course syllabi, and academic resources. Other implementations help students retrieve and summarize lecture notes, textbooks, and research materials. Additionally, several RAG-based tools focus on providing explanations and guidance for course-specific materials. The goal is to reduce the repetitive workload for instructors by incorporating an instructor-provided reference answer and grading rubric as an external knowledge base for short-answer grading [16].

Nevertheless, adapting RAG to education contexts remains a challenge. In the following sections, we detail what these challenges are, why they are important, and give a few examples of the existing research directions to address them. This review is important to understand the underlying problem that Q-RAG is trying to solve.

2.2.1 Challenges in Adapting RAG to Education

Retrieval Accuracy Recent educational RAG systems employ a variety of techniques to improve performance and tackle education-specific challenges. It is important to ensure retrieval accuracy since the relevance of the retrieved documents directly affects answer quality [15]. This issue is similar to RAG's general problem with query-to-chunk retrieval. Possibly, it is even more common in the education domain: students' questions are very often ambiguous while course resources are highly specific, causing a lower probability of matching during retrieval.

One approach is to tackle the user-side issues by dealing with the potential ambiguity in students' queries. This involves performing a more "personalized" retrieval: if the system has a model of the student's prior knowledge or misconceptions, it could tailor the documents it retrieves (for example, fetching an easier explanation or a prerequisite concept review if the student is a beginner). The use of a knowledge graph (KG) can help approximate personalization. Abu-Rasheed et al. use a domain-specific KG for learning recommendations: the chatbot uses the KG to understand which concepts the student is asking about and retrieves explanations linked to those nodes [17]. By navigating the graph of concepts, the system can clarify an ambiguous query by exploring related topics (e.g., if a question is too broad, the KG can help pinpoint a subtopic to focus on). This KG-backed retrieval acts as a form of structured disambiguation, guiding the LLM toward the intended context.

Additionally, query expansion and refinement techniques can be used to handle vague questions. If a student asks an ambiguous question, the system can rephrase or augment it with context. Recent research, such as Chan et al., shows that teaching the model to rewrite queries for clarity or to decompose complex questions into simpler sub-questions leads to better retrieval results [18]. In practice, an educational RAG might use the conversation history or known student profile to enrich the query (providing context like the current topic or the student's grade level) and utilize

hybrid retrieval (combining keyword-based and embedding-based search) to ensure that both exact terminology and broader concepts are considered [17]. These strategies collectively improve the chances that the retrieved documents closely match the student's information needs.

Another approach is to address issues on the system's side (the course materials) by improving their generalizability. Levonian et al. employ a variant of "parent retrieval," first identifying the relevant textbook section via dense similarity, then using that section for detailed prompting [15]. It is similar to a multi-stage retrieval process: a coarse initial search finds a broad topic or chapter, and then a refined search finds specific sub-sections. This multi-step approach helps the course materials to be represented by high-level abstracts, which are more robust against students' ambiguous queries.

While existing solutions to deal with retrieval accuracy show potential, they more or less follow the main directions identified previously in the general RAG domain: refining queries or enhancing document representations. Query expansion and rewriting mitigate ambiguity but rely on the assumption that rephrased queries will lead to better matches, but whether these reformulated queries are more aligned with documents is heavily context-dependent and may not even be true in every case. Similarly, improving course materials by adding another layer of representations can be beneficial, yet it can be challenging to find out how many layers of abstraction / different representations are enough to align with thousands of variations in queries. These limitations highlight a fundamental challenge: existing methods aim to work around the constraints of current retrieval paradigms rather than fundamentally rethinking how queries and documents should be connected.

Effective Human Involvement Another critical area is incorporating human-in-the-loop mechanisms to refine the RAG system over time. Without human intervention, the system will continue to make the same retrieval mistakes as RAG itself lacks a mechanism for correction. This issue is especially serious in educational contexts, where students often ask the same questions repeatedly, which emphasizes the need for course staff to be involved and provide correct answers. By integrating human feedback, the system can be continuously improved, ensuring reliable responses, which is especially important in education, where accuracy matters.

A simple approach is to introduce human intervention as a fall-back mechanism when retrieval fails. In cases where the AI might be unsure or go off-track, the system can defer to a human mentor (TA or teacher). Abu-Rasheed et al.'s mentoring chatbot supports a group-chat mode where a human mentor can join the conversation when needed [17]. The chatbot will involve the mentor if the query is beyond its scope, ensuring the student still gets a correct answer and allowing the AI to learn from the mentor's response for similar future queries.

Course staff can potentially guide RAG systems by curating the knowledge base and setting policies. For example, before deployment, the system designers might filter the document corpus to ensure quality and define prompt rules or templates based on pedagogy. In the mentoring chatbot, domain experts provided rules (e.g., preferred answer length, terminology to use or avoid) which are embedded into the prompt construction [17]. This kind of human-curated prompt engineering ensures the LLM's output adheres to educational guidelines.

Another solution is to integrate a feedback loop where users (students or instructors) indicate the usefulness or correctness of answers, and this data is used to improve the model. Bai et al. introduced Pistis RAG [19], which leverages structured human feedback for learning-to-rank: students can mark an answer as helpful or request a different response (e.g., "copy," "regenerate," or "dislike"), and the system treats these as signals to reorder and select retrieved content in the future. By training on such preference data, the RAG system learns to favor the documents and answers that align better with human expectations, thereby improving quality in subsequent interactions.

How to integrate human-in-the-loop components to RAG is not a question to be answered, as the exact methodology is designed to fit the specific demands of a course. The examples reviewed above only represent a subset among many other attempts to integrate human-in-the-loop components in educational RAG. Yet, one common issue we found for not only these works but also many other strategies is that they are either too simplistic or too complex. For instance, integrating system prompts as in Abu-Rasheed et al. [17] are far too simple, as this only helps the LLM to take into account limited information due to the limited context window. While the straightforward falling-back mechanism [17] makes sense, this essentially still allows poor retrieval to happen, delivering back the work to repeatedly correct these false answers to the course staff. On the other hand, learning-to-rank mechanisms introduced by Pistis RAG [19], offer a strong

approach but introduce many variables, such as determining the appropriate threshold for how much the RAG system should rely on feedback, or curating the feedback to filter out poor-quality ones.

2.3 Addressing Research Gaps and Existing Challenges

Through the literature review, we gained an understanding of the general and specific issues in adapting RAG to education. We identified two main challenges for Q-RAG to solve: addressing poor retrieval accuracy and integrating human-in-the-loop components.

For the first problem, as evidenced, existing research fails to account for the inherent issues of question-to-chunk matching. This is due to differences in the linguistic representations of a question and a chunk. Therefore, our focus should not be on 'forcing' alignment between these different elements, as that would mean trying to solve a fundamentally flawed problem.

On the other hand, while there are no right or wrong answers for the second issue, we can still find lessons from past work to formulate our solution. Most existing setups often involve too little human work to address anything, or require too much human work to function. An ideal solution should aim to prevent mistakes from happening while being flexible enough to allow varying levels of human involvement. It should give course staff the freedom to decide how many issues they want to address while still functioning sufficiently well without it.

To address this research gap, we introduce Q-RAG (Figure 3), our take on improving the usability of RAG chatbots not only in education but also in other closed-domain QA settings. This approach aims to improve the current question-to-chunk retrieval paradigm by utilizing question-question retrieval, ensuring that student queries are matched with semantically aligned, pre-indexed model questions that belong to the same semantic space. Additionally, its feedback-loop learning mechanism allows human annotators to refine incorrect retrievals on a case-by-case basis. This ensures the system learns to fix not a singular problem but a problem pattern, helping it to avoid poor retrieval for similar questions in the future. Moreover, our approach allows course staff to directly intervene in poor retrieval cases as needed. How much a RAG system improves depends on the level of human input, but addressing specific issues does not require a full team; individual staff members can choose which cases to handle based on their availability and priorities.

3 Methodology

3.1 Architecture Overview

As illustrated in Figure 3, Q-RAG features two main improvements to Naive RAG: *query-to-query retrieval* to enhance retrieval robustness, and *feedback-loop learning* to incorporate human feedback to continually correct errors.

The first module, *query-to-query retrieval*, utilizes an alternative way of similarity matching. More specifically, in addition to using chunks for indexing as Naive RAG, Q-RAG represents these chunks with *model questions*. These questions are examples of queries that are already answered using the information of their connected chunks. With this representation strategy, the retriever can directly compare input questions with the model questions in the knowledge base, thus narrowing down the search space to similar "problem patterns". As similar model questions are found, the system returns the connected chunks, which are solutions to these problem patterns. When complemented with query-to-chunk retrieval, this method helps to target retrieval towards chunks with answers to similar questions, reducing the likelihood of retrieving only contextually relevant but unhelpful information.

To formulate these model questions, we utilize the second module, *feedback-loop learning*. Essentially, this module allows an external annotator to "correct" the poor retrieval cases by providing the ground truth chunks as feedback. These ground truth chunks are then linked to the original input questions and appended to the database. The underlying idea is that similar questions in the future will be matched to these model questions and directed to the correct chunks. Through this mechanism, an annotator can directly intervene in any case, at any time, to prevent the system from repeating poor context retrieval.

To fully understand how Q-RAG improves from Naive RAG, let us consider a scenario introduced in Figure 3. In this case, the user's input leads to poor retrieval (incorrect chunks) and

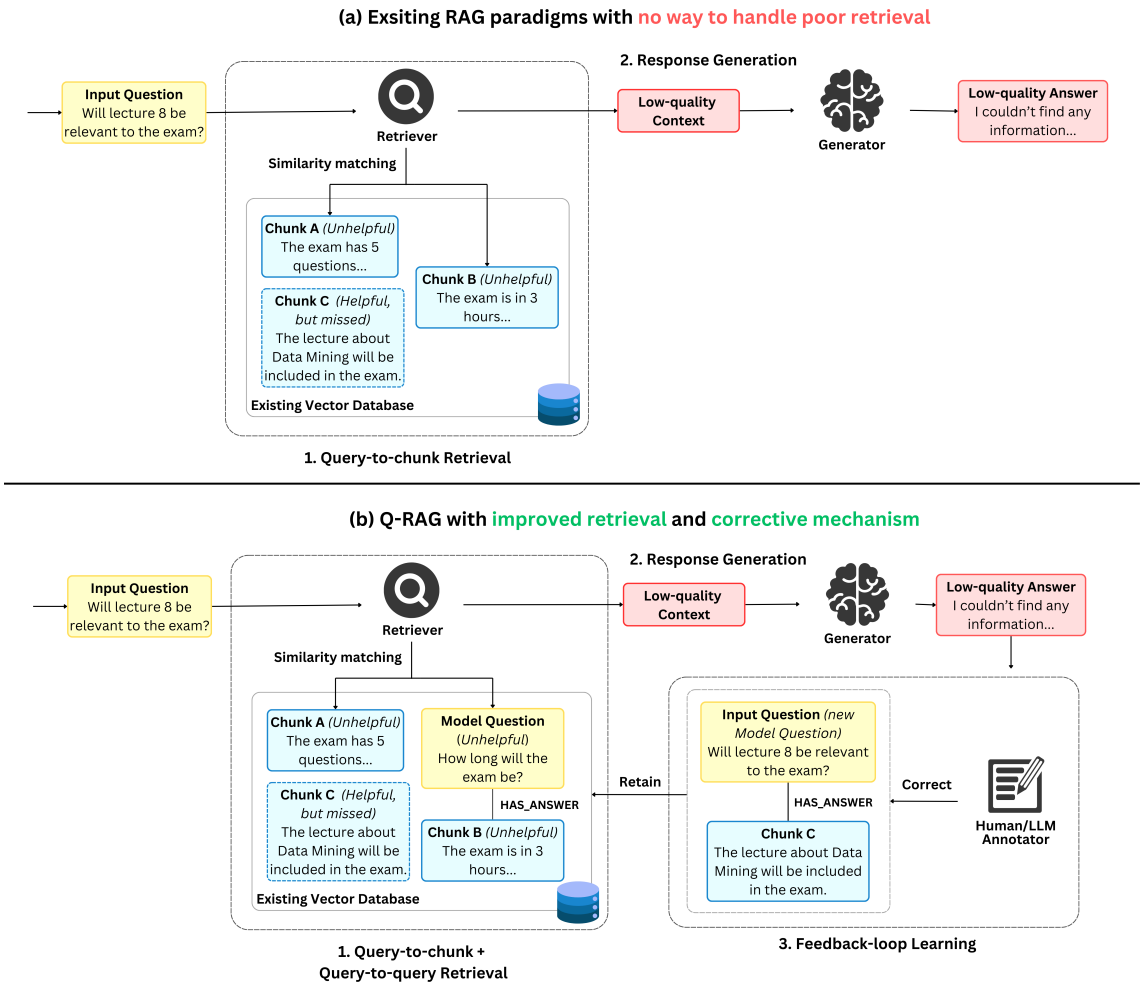

Figure 3: System architecture of RAG (a) versus Q-RAG (b). The main difference is (b1) *Query-to-query Retrieval*, where the retriever retrieves not only relevant chunks but also similar "model questions." It is assumed that these model questions are already linked to their corresponding chunks. (b3) *Feedback-loop Learning* involves an annotator providing corrected chunks for a query, which helps form query-chunk connections in the database. In the future, similar queries can be matched with these corrected chunks, which directly prevent poor retrieval from repeating.

results in a low-quality LLM response. Q-RAG addresses this by using the feedback-loop mechanism, where an annotator corrects the retrieval by identifying the right chunks and linking them to the original query. This introduces a new model question that similar queries can be matched with to retrieve the correct chunks. In the same process, the Naive RAG model fails to address retrieval errors since they have a linear nature. This represents the broad issue in existing solutions. Regardless of how many layers are added to improve retrieval, when the same problem arises, the model continues to make the same mistake, and the model (or the human) has to look for the fix again. These factors inspire Q-RAG to introduce a circular feedback loop that enables direct feedback and incorporates query-to-query retrieval to ensure that past corrections can be utilized during inference.

The following section will go deeper into the details of each stage of the model: (1) Retrieval, (2) Generation, and (3) Feedback, providing a mathematical formulation of the problem at each step.

3.2 Context Retrieval

During the retrieval process, Q-RAG utilizes both query-to-chunk and query-to-query matching. To explain the combination, one should remember that query-to-chunk is the basis of RAG. This

approach ensures that the retrieved chunks are relevant so the system can function normally. On the other hand, relying solely on the query-to-query method is impossible. Since query-to-query relies on an iterative learning process, a new, unexpected question can occur at any time and yield completely irrelevant retrieval results.

Query-to-chunk Retrieval In this stage, the retriever aims to search for relevant chunks d from an external knowledge source $\mathcal{D}$ to augment the LLM. This knowledge source could be materials, books, or other course-related resources. To selectively retrieve the relevant knowledge sources only, we introduce the query q to match with each $d \in \mathcal{D}$ using cosine similarity:

$$\text{similarity}(\mathbf{E}(q), \mathbf{E}(d)) = \frac{\mathbf{E}(q) \cdot \mathbf{E}(d)}{||\mathbf{E}(q)||\ ||\mathbf{E}(d)||}, \tag{1}$$

where $\mathbf{E}(x)$ represents the vector embedding for text x.

The process returns a set of k text chunks $\mathcal{D}'$ such that the similarities between q and these $d \in \mathcal{D}'$ are the largest among $\mathcal{D}$. This ensures that the retrieved chunks are semantically similar to the query, which could yield higher chances of the LLM discovering the key information to formulate the answer.

$$\mathcal{D}' = \underset{\mathcal{D}' \subset \mathcal{D}, |\mathcal{D}'|=k}{\text{argmin}} \sum_{d\ \in \mathcal{D}'} \text{similarity}(\mathbf{E}(q), \mathbf{E}(d)) \tag{2}$$

Query-to-query Retrieval Instead of matching queries directly to chunks, the query q will be matched to model questions q_{model} in another database $\mathcal{D}_q$ using the same similarity measure.

$$\mathcal{Q}' = \underset{\mathcal{Q}' \subset \mathcal{D}_q, |\mathcal{Q}'|=k}{\text{argmin}} \sum_{q_{\text{model}}\ \in \mathcal{Q}'} \text{similarity}(\mathbf{E}(q), \mathbf{E}(q_{\text{model}})) \tag{3}$$

In this database $\mathcal{D}_q$, each model question q_{model} are linked to their respective "solution chunks" $\mathcal{D}_{\text{model}}$ as $(q_{\text{model}}, \mathcal{D}_{\text{model}}) \in \mathcal{D}_q$. Therefore, by retrieving the relevant model questions, we can acquire the chunks linked to each of them, which can be merged to obtain the final set of relevant documents:

$$\mathcal{D}' = \bigcup_{q_{\text{model}} \in \mathcal{Q}', (q_{\text{model}}, \mathcal{D}_{\text{model}}) \in \mathcal{D}_q} \mathcal{D}_{\text{model}}, \tag{4}$$

The process returns a set of k queries, where each query is connected to its respective chunks. However, the final result will not be just chunks but a list of query-solution connections $\mathcal{D}' = \{(q_{\text{model}}, \mathcal{D}_{\text{model}}) \mid (q_{\text{model}}, \mathcal{D}_{\text{model}}) \in \mathcal{D}_q\}$. This ensures that each chunk is connected to a model question, serving as a "contextual header" that contains information on a chunk's usage.

Query-to-query retrieval ensures that there is a higher chance for user queries to be matched with similar problem patterns that already have a solution. Later, we will discover how these model query-chunk connections are generated as the result of feedback-loop learning.

Combining Retrieval Results In this framework, the retrieved chunks from each stage will be kept separate rather than merged into one large set. This is to maintain the association between certain chunks and their specific model questions. This explicitness is important to provide the LLM with more context to effectively utilize the given information. Since the ranking of documents matters, it is assumed that query-to-query retrieval results are ranked higher than those from query-to-chunk retrieval.

Before loading all the chunks in, we would like to eliminate duplicates of query-to-query chunks from query-to-chunk results, prioritizing information with a contextual header. The set of duplicates to be removed can be defined by:

$$\mathcal{D}_{\text{duplicates}} = \{d \in \mathcal{D}'_{\text{query-to-chunk}} \mid d \in \mathcal{D}'_{\text{query-to-query}}\} \tag{5}$$

In addition, it is required to return a ranked list of chunks for evaluation. For this purpose, we prioritize the context from query-to-query retrieval, putting them higher in the combined list than query-to-chunk retrieval results. Each item in the query-to-query retrieved subset is ranked by the higher similarity of the model question.

3.3 Response Generation

The generation stage is composed of two main steps: augmenting the LLM with the information and prompting the LLM to respond. The augmentation of the retrieved context is a form of *in-context learning*, which is a term referring to the process where the model dynamically adapts to the provided context without requiring updates to its parameters. This enables the LLM to incorporate external information to enhance its response generation while completely avoiding the need for fine-tuning.

During augmentation, the chunks retrieved $\mathcal{D}'_{\text{query-to-chunk}}, \mathcal{D}'_{\text{query-to-query}}$ will be combined with the original query q into a specialized prompt, detailed in Appendix B. This will be utilized to prompt the LLM $\mathcal{L}$ with pretrained parameters θ to generate the final response $\mathcal{R}$ using the retrieved context:

$$\mathcal{R} = \mathcal{L}(q \mid \mathcal{D}'_{\text{query-to-chunk}}, \mathcal{D}'_{\text{query-to-query}}, \theta) \tag{6}$$

The retrieval process allows the LLM to access dynamic contexts for a given query, which supplements the knowledge the inherent θ is missing. This is far more efficient and cheaper to maintain than fine-tuning an LLM to learn a course's materials, syllabus, or resources, making RAG-based paradigms highly suitable to low-resource settings.

3.4 Feedback-loop Learning

In certain situations, a student may prompt a question that leads to poor retrieval, which may stem from various specific reasons. The feedback-loop learning aims to address this by "learning" the corrected case with the help of an external annotator.

Let us assume that a certain query q' leads to poor retrieval $\mathcal{D}_{\text{poor}}$. An external annotator can intervene and provide the "corrected" chunks $\mathcal{D}_{\text{corrected}}$. The original query then becomes a new model query and can be connected to their respective solution chunks and appended to the database:

$$\mathcal{D}_q \leftarrow \mathcal{D}_q \cup \{(q_{\text{poor}}, \mathcal{D}_{\text{corrected}})\} \tag{7}$$

With this information retained, in the future, similar queries q can be matched with q_{poor} through query-to-query retrieval to obtain $\mathcal{D}_{\text{corrected}}$ as part of the retrieved data $\mathcal{D}'$:

$$q \sim q_{\text{poor}} \Rightarrow \mathcal{D}_{\text{corected}} \subseteq \mathcal{D}' \tag{8}$$

This approach ensures that queries leading to poor retrieval are addressed effectively. This is especially important in educational contexts, where students frequently ask similar questions, and it is crucial to address queries that lead to incorrect answers.

In theory, it is assumed that the annotator is an entity possessing the knowledge to find the correct chunks $\mathcal{D}_{\text{corrected}}$ to a certain query q. In practice, there are several ways to implement this. For instance, teaching assistants (TAs) could manually correct poor retrievals when the system encounters them, using a search engine to speed up the process of finding the correct chunks for a poor retrieval case. Alternatively, the process could be automated using an LLM annotator to pre-generate a QA dataset consisting of queries, reference answers, and corresponding chunks. This dataset can then be used to evaluate the system and identify queries that lead to poor retrieval. These queries can subsequently be linked to their respective reference chunks to augment the database. This approach ensures that the database is enriched with many resolved cases before deployment for student use.

4 Datasets

To evaluate the performance of Q-RAG, we used the 2024-published SyllabusQA dataset [20]. This dataset is suitable for our project as it focuses on the kind of syllabus and curriculum-related questions that university students commonly ask, mirroring the real-world use case we target with Q-RAG.

4.1 Data Description

SyllabusQA consists of over 13,000 question-answer pairs, each associated with one or more relevant context documents. The dataset spans over 150 university courses across multiple disciplines, including STEM, business, and social sciences. These questions are grounded in real-world course syllabi and students' information needs such as grading policies, deadlines, prerequisites, and course content [20].

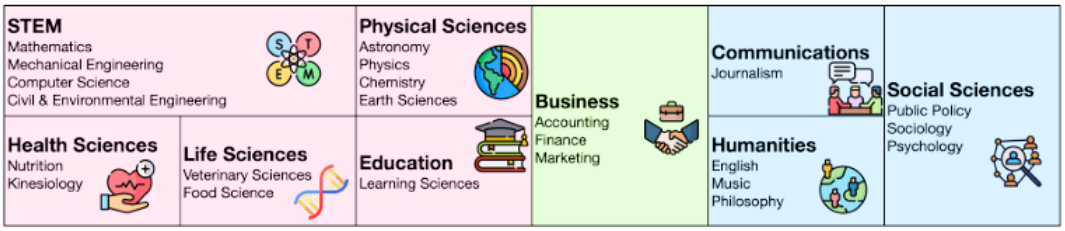

Figure 4: Domain diversity in SyllabusQA covering 36 majors. [20]

The dataset has three different splits: train, validation, and test sets. For this project, since no models are being trained (through parameter update), only the test set is taken into account. Overall, this set contains 632 data samples and has the following features:

- A natural language question that a student might ask realistically.
- A "golden" ground-truth answer, written by annotators based on syllabus content.
- At most 5 ground-truth supporting documents, typically drawn from the syllabus or related course pages.

Additionally, the dataset is designed to stress-test QA systems by including:

- Ambiguous or under-specified queries that require precise context to answer.
- Multi-hop questions, where the answer is not found in a single document chunk but requires integrating multiple dependent pieces of information.
- Hard negatives, making retrieval non-trivial even with strong embeddings.

4.2 Preprocessing

Preprocessing involved preparing both the original syllabus documents (the corpus) and the SyllabusQA dataset itself. These steps were crucial for aligning the data with our Q-RAG system's retrieval and generation requirements.

Corpus Processing: The corpus consists of the original syllabus documents from which questions and answers were derived.

- Remove non-UTF-8 characters from PDF-to-text conversions to ensure consistent text for accurate retrieval.
- Documents were divided into smaller, manageable text chunks, increasing retrieval precision by focusing on the most relevant information.
- Each chunk received identifiers: a parent ID links it back to its source document; a chunk ID indicates its sequence within the original document. This is done to ensure precise tracking of chunks, which is important for evaluation.

Dataset Processing: The following steps were performed to the test set to make sure the data covers only the desired scope:

- "Multi-reasoning" and "single-reasoning" questions were excluded from the dataset, as these depend solely on the language model's reasoning capabilities without requiring retrieval context.
- "No answer" (unanswerable) questions were removed from the scope of the test set, as these are only distractors and do not serve to evaluate retrieval-based methods.
- Original answer spans provided by dataset creators were realigned with our independently chunked documents. This is done to ensure accurate retrieval matching.

After filtering, the resulting test dataset only has 491 data samples left.

5 Experiments

5.1 Methodology

The objective of the experiment is to assess whether Q-RAG improves the overall performance compared to Naive RAG baselines, therefore helping to validate this proof of concept. Improvements in RAG systems can be measured in two key areas: the effectiveness of the retrieval process and the quality of the generated responses. This evaluation approach provides an understanding of the system's overall performance while allowing for a closer inspection of the individual components.

Figure 5 breaks down the main stages involved in the experiment. The experiment is abstracted into these steps so it is easier to understand the overall process:

- Test Set Construction: This involves preprocessing and loading the necessary components to form the evaluation dataset. This process includes curating a test set that consists of a question, the ground truth answer (the ideal LLM response), and the ground truth context (the necessary chunks required to answer the question).
- Results Generation: this stage prompts the RAG pipeline with the question to obtain the answer from LLM generation and the chunks from vector retrieval. These components will be collected and compared with the ground truth during the next stage.
- Metric Computation: the third stage utilizes the results generated from the previous stage to evaluate against the ground truth data from the test set. The results will produce metrics that give us insights into the performance of the retrieval and the generation process.
- Feedback Loop: The previous step helps identify poor retrieval cases through metrics. Using this information, we isolate the questions leading to poor retrieval and "correct" them by explicitly linking each question to its corresponding ground truth chunks. This simulates a feedback process where the annotator is assumed to have perfect knowledge of the relevant chunks. The constructed question-chunk pairs are then appended to the Q-RAG database.
- Test Set Reconstruction: Since the questions that previously resulted in poor retrieval are now incorporated into Q-RAG, evaluating the pipeline with the old data would no longer be meaningful. To maintain a fair evaluation, we simulate the emergence of new, similar questions by reformulating the old questions. Specifically, we use an LLM agent to generate rephrased versions of the original questions.
- Re-evaluation: The RAG systems (Q-RAG and baselines) will repeat step 1, 2, and 3 to obtain the final results on the rephrased test set.

To summarize, we first run the baseline RAG system through the test set to obtain the generated answer and retrieved documents. From these results, we use metrics to threshold and identify poor retrieval cases. Each question in the "poor-retrieval" set will be linked to its respective ground truth chunks and appended to Q-RAG's database. One can think of this as the "zero-iteration" to identify cases where Naive RAG would fail and produce the dataset to help Q-RAG learn from these errors.

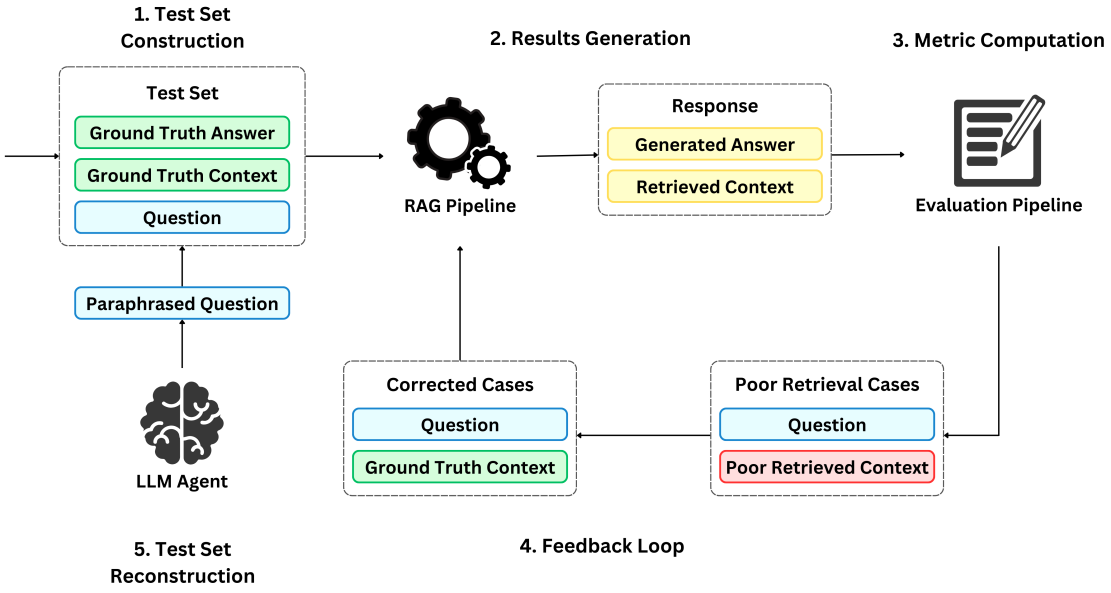

Figure 5: Design of the experiment pipeline

With these question-context pairs corrected, evaluating the RAG systems on the same test set would no longer be meaningful. To ensure a fair evaluation, we reconstruct the test set by reformulating the original questions using a specialized LLM prompt, given in Appendix D. This rephrasing process generates new variations of the original test questions while preserving their intent, simulating how new similar queries might appear in real-world scenarios. The modified test set is then used to re-evaluate the Q-RAG system and its baseline.

This process ensures that the Q-RAG pipeline can be learned before testing, similar to how the system could be "trained" to handle commonly seen questions before being used by students. The experiment creates a practical environment where the system is pushed to handle ambiguities and variations of the same question, which can help evaluate whether the system utilizes effectively the knowledge it has learned.

The specific details of the implementations are given in Appendix E. The appendix discusses the technology used to construct the pipeline and the relevant parameters set for each component.

5.2 Evaluation Metrics

5.2.1 RAGAs

Es et al. [21] developed a framework called Retrieval Augmented Generation Assessment (RAGAs) to evaluate RAG-based systems. This framework aims to measure the two important dimensions of RAG, the retriever and the generator. To achieve this, RAGAs utilizes an assigned LLM Judge to analyze the question, the ground-truth components, and the generated components to calculate the metrics.

The following metrics are chosen: Context Recall (CR), Context Precision (CP), and Faithfulness (FA). They all return a score ranging from 0 to 1, with higher values denoting better performance. In the experiment, the metrics are calculated for each entry in the test set. The average values of these scores are then computed to assess the performance of the model over the entire test set.

Context Recall (CR) CR measures the ability of the retriever to cover as many relevant documents as possible.

The metric uses string comparison to identify if the retrieved chunks are among the given ground truth contexts. This will determine whether a retrieved passage is relevant or not. The calculations follow the classic definition of document recall:

$$\text{CR} = \frac{|\text{Number of relevant contexts retrieved}|}{|\text{Total number of relevant contexts}|}$$

In the context of the experiment, this essentially computes the portion of the ground truth chunks that have been retrieved. This metric can directly measure the retriever component's ability to capture every context needed to answer the question.

Context Precision (CP) CP measures whether the ground-truth documents present in the retrieved contexts are ranked higher or not. In an ideal scenario, the relevant documents should be at the top ranks of the retrieval result.

To compute the metric, CP uses a similarity score with a threshold to determine whether the documents in the ground truth are relevant to the document at each rank i ($0 \leq i \leq k$) among the top-i documents. If the document at rank k is relevant, the system returns a verdict $v_i = 1$, or $v_i = 0$ otherwise. For each rank i, RAGAs computes the Precision_i, which reflects the proportion of relevant documents (True Positives_i) among all documents so far ($\text{True Positives}_i + \text{False Positives}_k = i$). The final CP score is determined by the average Precision_k for all ranks k.

$$\text{CP}_k = \frac{\sum_{i=1}^{k}(\text{Precision}_i \times v_i)}{\text{Number of relevant documents in the top } k \text{ results}}$$

$$\text{Precision}_i = \frac{\text{True Positives}_i}{\text{True Positives}_i + \text{False Positives}_i}$$

CP can evaluate whether the retriever component ranks the ground-truth chunks higher among the retrieved results. This helps to assess the ranking precision of the retriever component.

Faithfulness (FA) FA computes the consistency of the answer to the provided context. According to RAGAs' definition, the generated answer is considered faithful if claims made in the answer can be inferred from the context.

During computation, an LLM is prompted to deconstruct the answer into claims. For each claim, the LLM verifies whether this can be deduced from the context. RAGAs then count the number of claims and perform calculations according to the formula:

$$\text{FA} = \frac{|\text{Claims in the answer that can be inferred from context}|}{|\text{Total number of claims in the answer}|}$$

FA measures the generator's faithfulness and consistency to the retrieved context, which helps to determine whether the LLM component is prone to hallucinations or not. This is especially important in a knowledge-intensive task like education, where the answers delivered to students are expected to be consistent to course materials.

5.2.2 G-Eval

Liu et al. [22] introduced G-Eval, a complex framework for automatically evaluating generated answers using large language models (LLMs) as judges. Unlike traditional metrics such as ROUGE [23] and BLEU [24], which rely on surface-level lexical overlap, G-Eval is designed specifically for open-ended, free-form responses. It employs prompt-based instructions to assess the quality of generated content across multiple dimensions such as correctness, relevance, coherence, and fluency.

Answer Correctness (AC) Appendix C shows the set of criteria designed to measure the factual correctness of the generated answers compared to the ground truth ones. For this task, G-Eval presents the LLM judge with the original question, the generated answer, and a reference answer, and prompts the LLM to identify factual alignment between the answers. Given a set of user-defined criteria, the LLM will generate a chain-of-thought (CoT) as an action sequence to evaluate and generate the verdict. The final verdict contains a score from 0 to 1 reflecting the criteria and the detailed reasoning explaining its decision for validation.

AC is one of the standard approaches in quantifying the performance of LLM agents in QA tasks [25]. In our use case, the metric helps assess the performance of the LLM component in terms of correctly presenting the necessary facts to answer a student's question.

5.3 Baselines

The baseline system utilized to run the zero iteration and to compare with Q-RAG is a Naive RAG system. It follows the design of Figure 2. Besides the clear architectural difference, the baseline model utilizes the same parameters and model type for each component in Q-RAG, ensuring a fair comparison.

6 Results

6.1 Zero-iteration Results

As shown in Table 1, the Naive RAG system was able to obtain the average recall score of 0.77 on its first run over the test set. In other words, the model fails to find enough evidence needed to answer a question 23% of the time.

Models	CR
Naive RAG	0.77

Table 1: Average CR over the test set of Naive RAG

This demonstrates that poor retrieval is quite common, particularly in educational QA settings where similar questions are frequently repeated. The zero-iteration setup narrows the test set to cases where Naive RAG is likely to fail. This allows us to identify the queries that result in poor retrieval, which can then be used to construct the training dataset for Q-RAG.

6.2 Test Reconstruction Results

Figure 6 provides a subset of the reconstructed queries for qualitative review. It is possible to observe that the rephrased questions have a relatively strong difference from their original forms while maintaining the original intent.

Question	Rephrased Question
Is there any makeups on home work?	Can homework be resubmitted or compensated for?
What sort of help can I get if I am having trouble with the material?	What types of assistance are available if I'm struggling with the course content?
What is the proposed area of instruction?	What are the suggested subjects for teaching?
What are the assignments?	Which tasks are required to be completed?
What is the importance of Mid-terms in this class?	Why are midterm exams significant in this course?
How many areas will I be graded on in this class?	What are the different components upon which my performance will be assessed in this class?
What is the required reading?	Which books and articles must be read for the course?
What are the required readings for the course?	What are the mandatory texts for this class?
What are the learning formats in this class?	What types of learning opportunities are available in this
Are the exams open book or closed book?	Are the exams available for use with or without reference
In order to succeed in this course, what are some key skills we need to master?	What essential abilities should we hone to excel in this class?
Are candidates permitted to serve as substitute teachers?	Can candidates act as temporary instructors?
What are some of the key points of this class?	What are the main topics covered in this course?
What are the points for the final?	What is the total point value for the final exam?
Can you give me details on the homework for this class?	What are the specifics regarding the assignments for this
What are some additional readings I can look into for this	What supplementary materials can I explore for this course?

Figure 6: A sample of the rephrased questions

To quantify the difference between the reconstructed and original sets, one can analyze the similarity scores between each question pair. Overall, the average similarity between the pairs is around 82.5%, which is high enough to ensure semantic consistency while introducing lexical variations.

As shown in Figure 7, the majority of the question pairs fail within the 75-90% range, indicating that most rephrased variants retain strong semantic alignment with their originals. The long tail of lower similarity pairs represents the more challenging questions where phrasing differences are substantial, which may help to evaluate the robustness to varied students' expressions better.

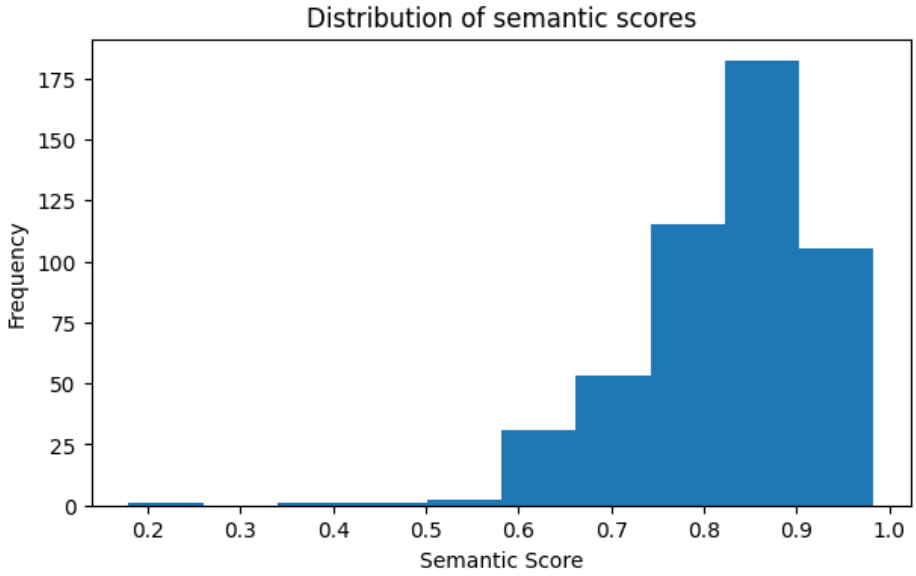

Figure 7: Distribution of similarity scores between original and rephrased questions

6.3 Main Results

Table 2 shows that Q-RAG enhances the performance of RAG on all aspects, confirming its effectiveness in this QA task.

Models	CR	CP	FA	AC
Q-RAG	**0.98**	**0.62**	**0.89**	**0.72**
Naive RAG	0.77	0.43	0.87	0.65

Table 2: Average evaluation scores over the rephrased test set of Q-RAG and Naive RAG

On the retrieval side, the CR metric indicates that Q-RAG outperforms Naive RAG in capturing ground-truth chunks. Meanwhile, the CP score suggests that the system prioritizes ground-truth chunks higher in the retrieval results. These suggest that the trained Q-RAG system is highly robust against the rephrased questions and it is using the learned knowledge effectively. Regarding the generation aspect, FA remains consistently high across methods, indicating that Q-RAG, despite introducing more documents (and thus potentially more noise), does not negatively impact model reliability. The improved performance in answer quality is also reflected in Q-RAG's higher answer correctness (AC).

Beyond the average scores shown in Table 2, Table 3 summarizes metric differences for each query when moving from Naive RAG to Q-RAG on the same rephrased test set:

Question type	ΔCR	ΔCP	ΔFA	ΔAC
multi factual	**+0.25**	**+0.24**	**+0.02**	**+0.07**
single factual	+0.20	+0.20	−0.01	+0.06
summarization	+0.13	+0.12	+0.00	+0.03
yes/no	+0.20	+0.20	+0.00	+0.02

Table 3: Mean score improvements from Naive RAG to Q-RAG by question type

One can observe that all question types benefit from Q-RAG, especially multi-factual and single-factual queries. These results suggest Q-RAG is particularly strong at fixing retrieval for more well-defined and targeted questions. Meanwhile, faithfulness scores remain mostly stable, while correctness improves modestly.

The metric distributions in Figure 8 can reveal nuanced patterns in the models' performances.

CP fluctuates widely, especially for ambiguous or complex queries, showing that the RAG models tend to mix irrelevant chunks in retrieval results in certain cases. Correctness (AC) also varies; this is understandable, as correct retrieval does not guarantee accurate final answers. More robust prompt design or refined chunk usage could help the LLM address these issues. Meanwhile, CR and FA are mostly in high-score regions, indicating that the performances across all models are relatively stable in capturing correct documents and minimizing hallucinations, respectively.

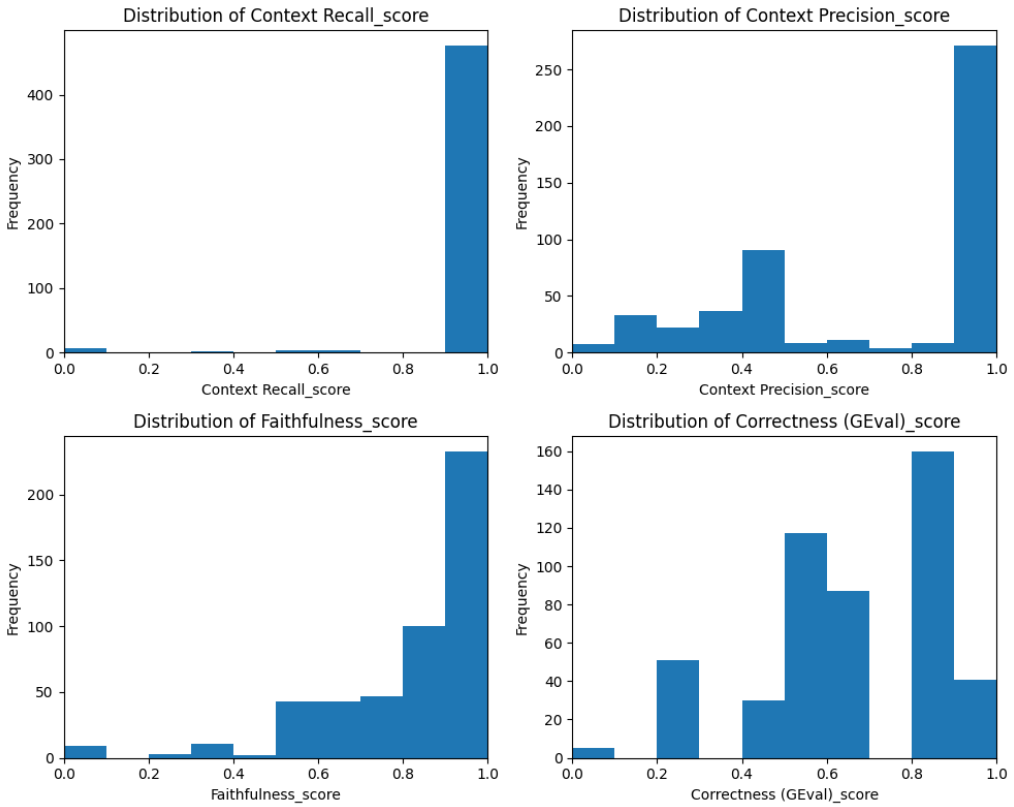

Figure 8: Distribution of metric scores for both models (Q-RAG and Naive RAG)

Figure 9 describes the average metric values across question types. They help to show clear patterns: the systems performs consistently strong with single-factual queries due to their simple, one-hop structure; yes/no combine top recall with the lowest correctness, showing the challenge of forming a definitive binary conclusion even when relevant text is retrieved. Lastly, summarization and multi-factual queries both have moderate correctness despite strong recall, highlighting the difficulty in synthesizing multi-chunk context into coherent, complete answers. This result is expected and shows a flaw that Q-RAG can focus on improving.

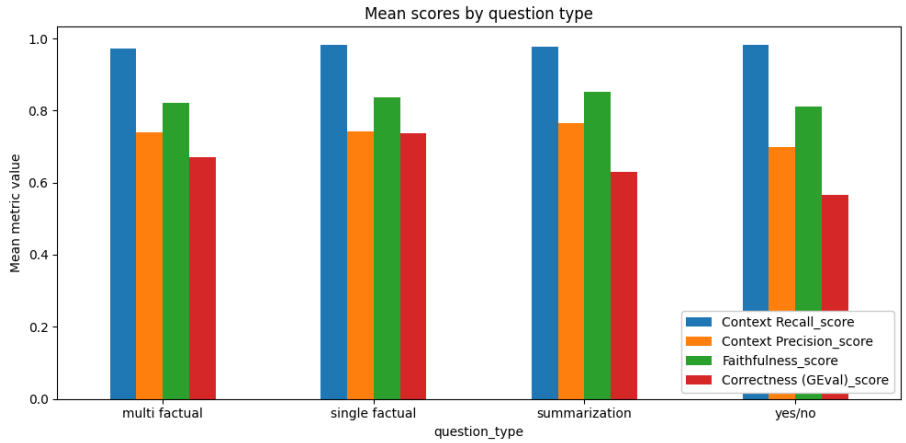

Figure 9: Mean metric values for both models (Q-RAG and Naive RAG) across all question types: multi-factual, single-factual, summarization, and yes/no.

To analyze the error cases, we identified queries leading to particularly low scores (outliers). Using retrieval recall as the threshold (CR < 0.30), we can extract these outliers in both Naive RAG and Q-RAG. The frequencies of outliers among all question types are displayed in Table 4.

Model	**Total outliers**	**Multi-factual**	**Single-factual**	**Summarization**	**Yes/no**
Naive RAG	89	30	25	9	25
Q-RAG	7	2	2	1	2

Table 4: Outlier frequency for both models (Q-RAG and Naive RAG) among question types

It can be observed that Q-RAG nearly eliminates low-recall outliers, reducing failure cases from 89 to just 7. Most remaining outliers are likely to be multi-factual or ambiguous, which are often not easily solvable.

Despite the improved performance, Q-RAG remains highly efficient. Table 5 shows that there are virtually no differences in terms of average response time. The only downside is that Q-RAG generally utilizes more context (due to the dual retrieval approach), which can lead to more token usage in LLMs and more retrieval calls. But as one can observe, this slight increase in time is very minimal, making it a practical system to deploy in real-time educational tools.

Models	Time/Query (sec.)
Q-RAG	7.01
Naive RAG	6.42

Table 5: The elapsed time per query for each model

In conclusion, the feedback loop mechanism has successfully allowed the QA system to handle past mistakes effectively, as evidenced by the significant performance increase. One can also see that query-to-query retrieval is more effective than query-to-chunk retrieval, as it better captures the correct answers (CR) while prioritizing relevant information (CP). The results of the experiment have helped to validate Q-RAG in the education QA settings.

7 Conclusions

In this paper, we introduced Q-RAG, an extension of the standard RAG paradigm that incorporates an alternative retrieval approach and human-in-the-loop intervention. Through an experiment in the education setting, we demonstrated that Q-RAG not only addresses the limitations of traditional chunk-based retrieval but also leverages feedback to iteratively refine retrieval and avoid making the same errors. These findings highlight Q-RAG's potential as a more reliable and

adaptive QA system, particularly in domains where human intervention is essential. By validating Q-RAG as a proof of concept for an improved RAG-based chatbot system in education QA, this work has achieved the goals set out initially.

A key limitation of this research arises from the very mechanism that makes Q-RAG powerful: the feedback loop. Once a case has been corrected, Q-RAG essentially achieves "perfect retrieval" for that exact or highly similar query in future sessions. While this is beneficial for practical usage (students asking the same question get the correct context), it can cause biases that artificially inflate performance metrics during evaluation. Another limitation lies in the challenge of re-processing documents into chunks while preserving question-chunk links when the course content is updated. Additionally, the resources available for our experiments were limited, resulting in a small-scale evaluation. The evaluation of Q-RAG could significantly benefit from studies involving real-world users.

There are several promising directions for future research building on Q-RAG. One promising idea is to utilize a chain-of-thought (CoT) [26] reasoning or ReAct approach [27] to improve generalization for multi-hop queries. A core assumption of this paper is that annotators always possess the ground truth for every question. However, in real-world settings, this is not always the case, and human error is inevitable. More research could be done to explore ways to robustly introduce annotators into the feedback mechanism of Q-RAG. Additionally, future work could explore the development of user-friendly interfaces to facilitate Q-RAG's deployment across different course settings—for example, tools for collecting low-performance cases, platforms for integration of TA feedback, and curating course materials suitable for the chatbot.

8 Division of Labor

- Hung Nguyen constructed the RAG models, Q-RAG and their baselines. Hung also built the evaluation pipeline to assess the results.
- My Nguyen crawled and preprocessed the dataset. My also analyzed the results and concluded the paper.

9 Acknowledgement

We would like to thank our TA, Tamas Grosz. He helped us validate the idea and plan the schedule during the initial stages of the research. We would also like to thank Aalto IT for accepting our case and helping us set up the Azure OpenAI pipeline, which has been a cornerstone in building and evaluating the RAG models.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.

[2] Dominik Thüs, Sarah Malone, and Roland Brünken. Exploring generative ai in higher education: a rag system to enhance student engagement with scientific literature. *Frontiers in Psychology*, 15, 2024.

[3] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system, 2024.

[4] Garima Agrawal, Tharindu Kumarage, Zeyad Alghamdi, and Huan Liu. Mindful-rag: A study of points of failure in retrieval augmented generation, 2024.

[5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler,

Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[6] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, and Suranga Nanayakkara. Fine-tune the entire rag architecture (including dpr retriever) for question-answering, 2021.

[7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.

[8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.

[9] Adrian H. Raudaschl. The next frontier of search: Retrieval augmented generation meets reciprocal rank fusion and generated queries, 2023.

[10] Zackary Rackauckas. Rag-fusion: A new take on retrieval augmented generation. *International Journal on Natural Language Computing*, 13(1):37–47, February 2024.

[11] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020.

[12] Haoyu Zhang, Jun Liu, Zhenhua Zhu, Shulin Zeng, Maojia Sheng, Tao Yang, Guohao Dai, and Yu Wang. Efficient and effective retrieval of dense-sparse hybrid vectors using graph-based approximate nearest neighbor search, 2024.

[13] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333–389, April 2009.

[14] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024.

[15] Zachary Levonian, Chenglu Li, Wangda Zhu, Anoushka Gade, Owen Henkel, Millie-Ellen Postle, and Wanli Xing. Retrieval-augmented generation to improve math question-answering: Trade-offs between groundedness and human preference, 2023.

[16] Hefei Qiu, Brian White, Ashley Ding, Reinaldo Costa, Ali Hachem, Wei Ding, and Ping Chen. Stella: A structured grading system using llms with rag, 2025.

[17] Hasan Abu-Rasheed, Mohamad Hussam Abdulsalam, Christian Weber, and Madjid Fathi. Supporting student decisions on learning recommendations: An llm-based chatbot with knowledge graph contextualization for conversational explainability and mentoring, 2024.

[18] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. Rq-rag: Learning to refine queries for retrieval augmented generation, 2024.

[19] Yu Bai, Yukai Miao, Li Chen, Dawei Wang, Dan Li, Yanyu Ren, Hongtao Xie, Ce Yang, and Xuhui Cai. Pistis-rag: Enhancing retrieval-augmented generation with human feedback, 2024.

[20] Nigel Fernandez, Alexander Scarlatos, and Andrew Lan. Syllabusqa: A course logistics question answering dataset, 2024.

[21] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation, 2023.

[22] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: NLG evaluation using gpt-4 with better human alignment. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, Singapore, December 2023. Association for Computational Linguistics.

[23] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.

[24] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In Pierre Isabelle, Eugene Charniak, and Dekang Lin, editors, *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA, July 2002. Association for Computational Linguistics.

[25] Yuxia Wang, Minghan Wang, Muhammad Arslan Manzoor, Fei Liu, Georgi Georgiev, Rocktim Jyoti Das, and Preslav Nakov. Factuality of large language models: A survey, 2024.

[26] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

[27] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.

[28] LangChain. Langchain: Applications that can reason. https://www.langchain.com/, 2024. Powered by LangChain.

[29] Qdrant. Qdrant - high-performance vector search at scale, 2025. Accessed: 2025-03-18.

[30] Inc. Docker. Docker: Accelerated container application development, 2025.

[31] Nils Reimers and Iryna Gurevych. Sentence-bert pretrained models, 2024.

[32] Aalto University. FAQ - Productised AI and Azure OpenAI. https://www.aalto.fi/en/services/faq-productised-ai-and-azure-openai, 2024. On this page you will find frequently asked questions and their answers for productised AI and Azure OpenAI.

[33] Confident AI Inc. Deepeval: The open-source llm evaluation framework. https://docs.confident-ai.com/, 2025. Copyright © 2025 Confident AI Inc. Built with and confidence.

Appendix A Codebase

The codebase can be found in GitHub.

Appendix B LLM Agent Prompt Design

```
System Prompt:
Your role is to help students by providing accurate and concise
    answers based on the given context.
- Use the provided documents to formulate your response.
- Do not mention the presence of these documents in your response.
    Just extract relevant facts you can use to build up your answer.

User Prompt:
# Question: {query}

# Similar Answered Questions:
Question: {model_question_1}
Context: {linked_context_1}
...
Question: {model_question_k}
Context: {model_question_k}
```

```
# Other Related Documents:
Context: {context_1}
...
Context: {context_k}
```

Listing 1: Prompt template for augmentation

Appendix C LLM Judge Criteria

```
System Prompt:
- Check whether the facts in 'actual output' contradict any facts in
    'expected output'.
- You should also heavily penalize omission of detail.
- You should not penalize redundant details in 'actual output'.
- Vague language, or contradicting OPINIONS, are OK.
```

Listing 2: Prompt template for LLM Judge

Appendix D Question-Rephrasing Prompt Design

```
System Prompt:
You are provided with a QUESTION from a student. Your task is to
    transform the QUESTION into a REPHRASED QUESTION by rephrasing it
     differently.
- Do a strong paraphrasing. However, make sure to leave enough
    information in the REPHRASED QUESTION so that the ANSWER to it do
     not change.
- Do not add any redundant text to your response besides the
    rephrased question. For example, do not add the "REPHRASED
    QUESTION" text to your response.

User Prompt:
QUESTION: {question}
ANSWER: {ground_truth_answer}
```

Listing 3: Prompt template for question rephrasing

Appendix E Implementation Details

The system was built using LangChain [28], a framework powered for LLM Agent construction. The database was supported by Qdrant [29], a database optimized for vector searching. An instance of the database was created and hosted locally using Docker [30]. The embedding model (`sentence-transformers/all-MiniLM-L6-v2`), a lightweight and competitive model for retrieval tasks, is provided by Sentence Transformers [31]. The LLM was hosted by Aalto's Azure OpenAI service [32]. Metrics calculation and evaluation were conducted using the DeepEval library [33].

Parameter	Value
RAG's top-k (chunks to retrieve)	5
Q-RAG's top-k (questions to retrieve)	1
Q-RAG's similarity threshold	0.5
Chunk Size	1500
Chunk Overlap	1000

Table 6: Vector Database

Parameter	Value
Model	gpt-4o
Temperature	0.1
Top-p	0.8

Table 7: LLM Agent Parameters

Parameter	Value
Model	gpt-3.5-turbo
Temperature	0.

Table 8: LLM Judge Parameters

Parameter	Value
Model	sentence-transformers/all-MiniLM-L6-v2
Vector Dimension	384
Device	CPU

Table 9: Embedding Model Parameters